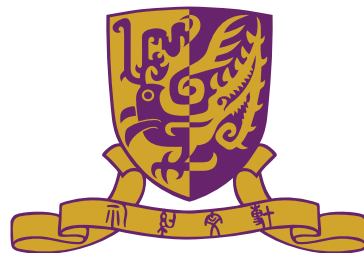


# DDA5001 Machine Learning

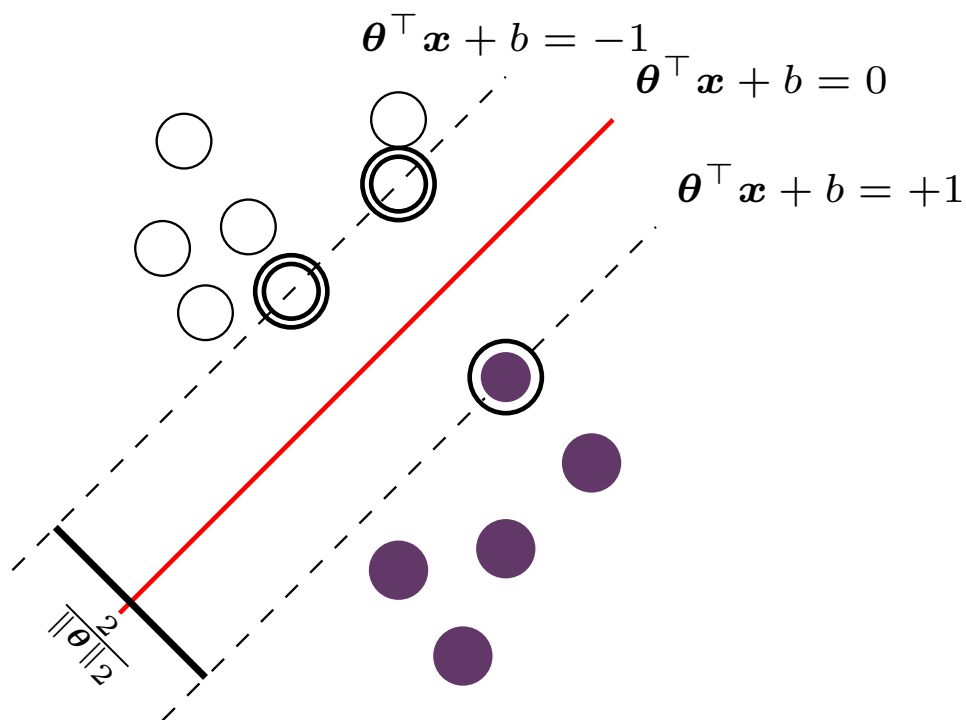
## Kernel Method

**Xiao Li**

School of Data Science  
The Chinese University of Hong Kong, Shenzhen



# Recap: Hard-margin SVM



Hard-margin SVM:

$$\underset{\theta \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \|\theta\|_2^2$$

$$\text{subject to } y_i(\theta^\top \mathbf{x}_i + b) \geq 1, \quad i = 1, \dots, n$$

Hard-margin SVM can be regarded as: Choose a classifier from all possible perceptron classifiers that has the largest margin.

# Recap: Regularization View and Modified VC Analysis

- ▶ SVM is a linear classifier robust to noise. Such a robustness can be interpreted as: SVM is better '**regularized**', which corresponds to the **weight decay** term in the objective function.
- ▶ SVM maximizes margin, which also helps reduce the generalization error by using **margin  $\rho$**  rather than number of parameters  $d + 1$ .

Theorem: VC dimension of margin- $\rho$  hyperplanes

Suppose the input space is the ball of radius  $R$  in  $\mathbb{R}^d$ , that is  $\|\mathbf{x}\| \leq R$ . Then,

$$d_{\text{VC}}(\rho) \leq \lceil R^2 / \rho^2 \rceil + 1.$$

- ▶ Since its VC dimension is at most  $d + 1$ . Thus, we can have

$$d_{\text{VC}}(\rho) \leq \min\{\lceil R^2 / \rho^2 \rceil, d\} + 1.$$

- ▶ The bound suggests that the margin  $\rho$  can be used to control the model complexity. Hence, seeking for a **max margin (max  $\rho$ )** can lead to better  $\text{Er}_{\text{out}}$ .

# Recap: Soft-margin SVM

Hinge loss:

$$h(\boldsymbol{\theta}; \mathbf{x}_i, y_i) = \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b))$$

- ▶ The hinge loss is a **relaxation** of the perceptron hard constraints.

The **soft-margin** SVM is to solve

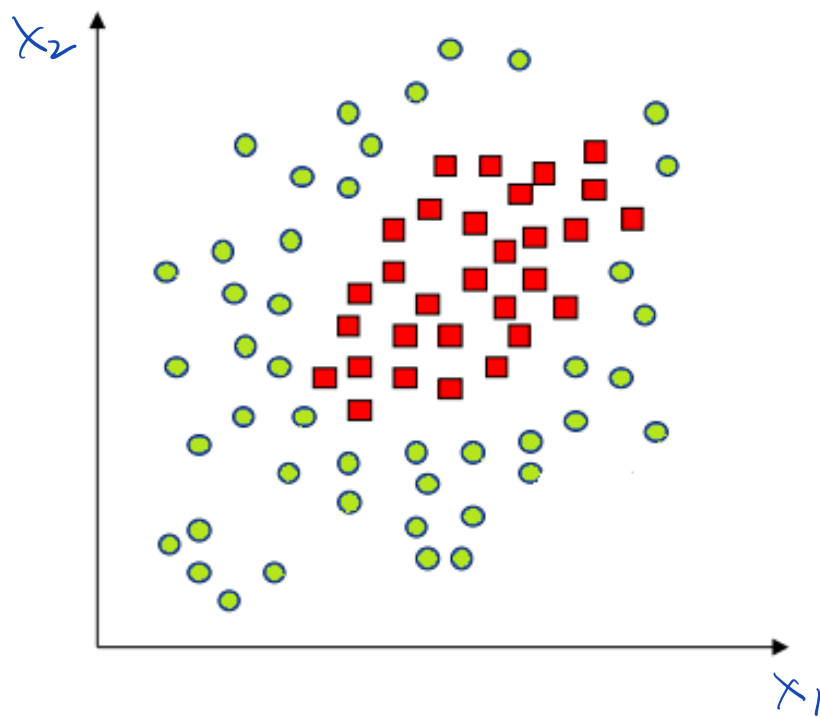
$$\underset{\boldsymbol{\theta} \in \mathbb{R}^d, b \in \mathbb{R}}{\text{minimize}} \quad \frac{1}{n} \sum_{i=1}^n \max(0, 1 - y_i(\boldsymbol{\theta}^\top \mathbf{x}_i + b)) + \lambda \|\boldsymbol{\theta}\|_2^2$$

- ▶  $\lambda > 0$  determines the trade-off between increasing the margin and ensuring that  $\mathbf{x}_i$  lies on the correct side of the margin.
- ▶ If the data is linearly separable, then we can choose a sufficiently small  $\lambda$  to let the soft-margin SVM works the same as hard-margin SVM.
- ▶ If data is not linearly separable, soft-margin SVM can still provide a meaningful classifier.

## Kernel Method

# The Limitation of Linear Model

- ▶ We have studied Perceptron, LS, LR, SVM. They are all **linear models**.
- ▶ Sometimes linear model is restrictive.



- ▶ Can you find a linear model that classify well these two classes?

## Nonlinear Transform

# The Underlying Idea

- ▶ No. We cannot find a linear model in **dimension  $d$** .

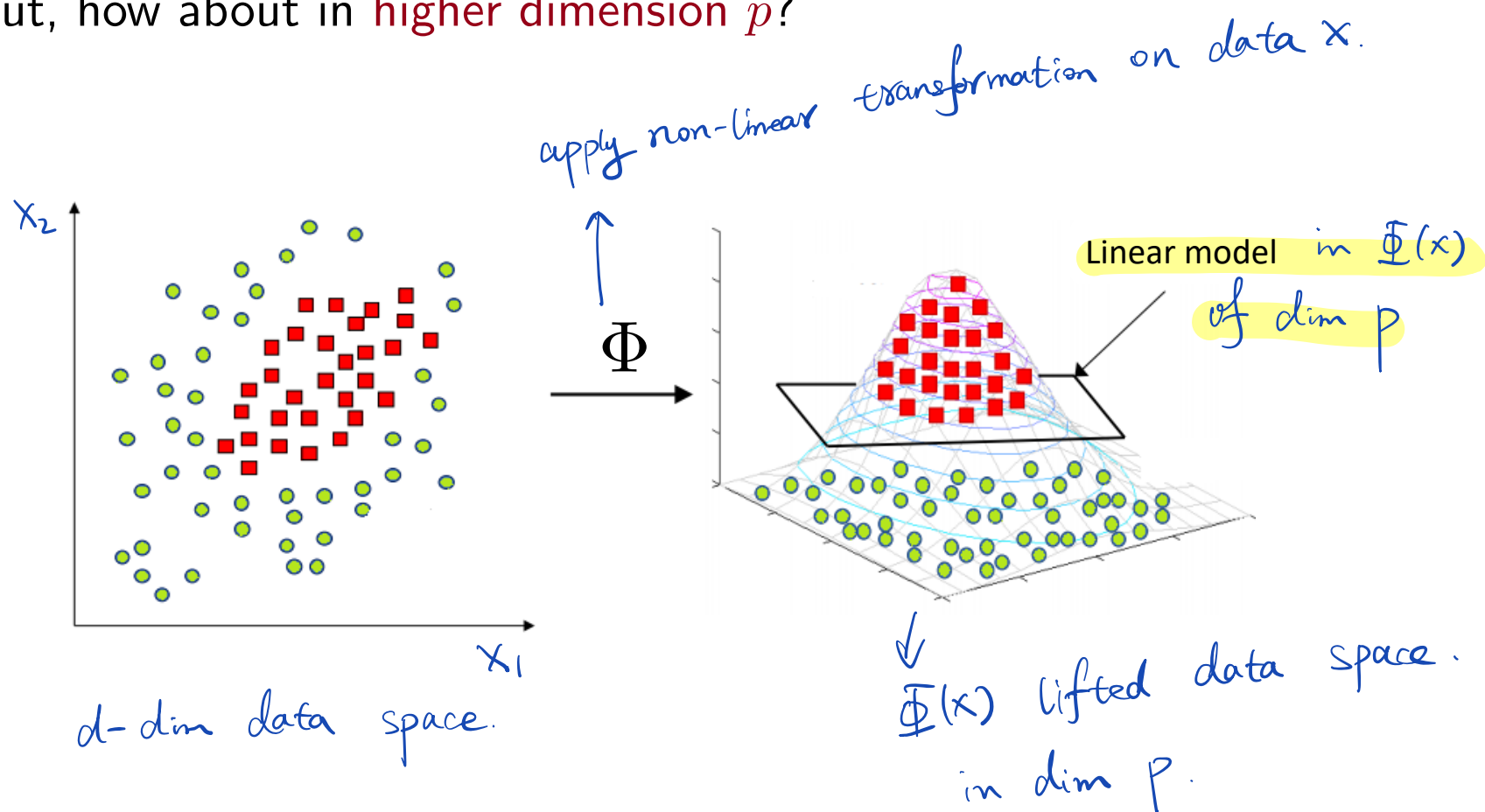


# The Underlying Idea

- ▶ No. We cannot find a linear model in dimension  $d$ .
- ▶ But, how about in higher dimension  $p$ ?

# The Underlying Idea

- ▶ No. We cannot find a linear model in dimension  $d$ .
- ▶ But, how about in higher dimension  $p$ ?



# Nonlinear Transformation to Data

Transformation function:

$$\Phi(\mathbf{x}) : \mathbb{R}^d \rightarrow \mathbb{R}^p, \quad d < p.$$

Example in 1d:

$$x \in \mathbb{R} \xrightarrow{\Phi} \Phi(x) = \begin{bmatrix} x \\ x^2 \\ \vdots \\ x^p \end{bmatrix} \in \mathbb{R}^p$$



Then, try a linear model on the transformed data  $\{\Phi(\mathbf{x}_1), \dots, \Phi(\mathbf{x}_n)\}$ .

# Generalized Linear Model (Classification Case)

Let us take binary classification as an example. The same idea applies to regression.

► Original training data:  $\{(\overset{\substack{\uparrow \\ \mathbb{R}^d}}{\boldsymbol{x}_i}, y_i)\}_{i=1}^n$ .

► Transformed training data:

$$\{(\Phi(\boldsymbol{x}_i), y_i)\}_{i=1}^n.$$

$\downarrow$   
 $\mathbb{R}^p$

# Generalized Linear Model (Classification Case)

Let us take binary classification as an example. The same idea applies to regression.

► Original training data:  $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ .

► Transformed training data:

$$\{(\Phi(\mathbf{x}_i), y_i)\}_{i=1}^n.$$

► It becomes linearly separable even with linear model in  $\boldsymbol{\theta} \in \mathbb{R}^p$

$$y_i = \text{sign}(\Phi(\mathbf{x}_i)^\top \boldsymbol{\theta}), \quad \forall i = 1, \dots, n.$$

↪ Then applying Perceptron, LR, or SVM to learn  $\hat{\boldsymbol{\theta}}$  based on  $\{(\Phi(\mathbf{x}_i), y_i)\}_{i=1}^n$ .

# Issues of Nonlinear Transformation

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \xRightarrow{\Phi} \Phi(\mathbf{x}) = \begin{bmatrix} \Phi^{(1)}(\mathbf{x}) \\ \Phi^{(2)}(\mathbf{x}) \\ \vdots \\ \Phi^{(p)}(\mathbf{x}) \end{bmatrix}$$

where  $p > d$ .

- ▶ **Drawback I:** We can always find a separating hyperplane, by letting  $p > n$ .
  - However, this leads to **overfitting**.
  - Fortunately, this issue can be tackled by using **regularization**.
- ▶ **Drawback II:** Increasing computational load with increasing  $p$  (especially when  $p \rightarrow \infty$ ).

# Issues of Nonlinear Transformation

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \xRightarrow{\Phi} \Phi(\mathbf{x}) = \begin{bmatrix} \Phi^{(1)}(\mathbf{x}) \\ \Phi^{(2)}(\mathbf{x}) \\ \vdots \\ \Phi^{(p)}(\mathbf{x}) \end{bmatrix}$$

where  $p > d$ .

- ▶ **Drawback I:** We can always find a separating hyperplane, by letting  $p > n$ .
  - However, this leads to **overfitting**.
  - Fortunately, this issue can be tackled by using **regularization**.
- ▶ **Drawback II:** Increasing computational load with increasing  $p$  (especially when  $p \rightarrow \infty$ ).  $\rightsquigarrow$  This issue can be solved by **kernel method / trick**.

## Kernel Method / Trick

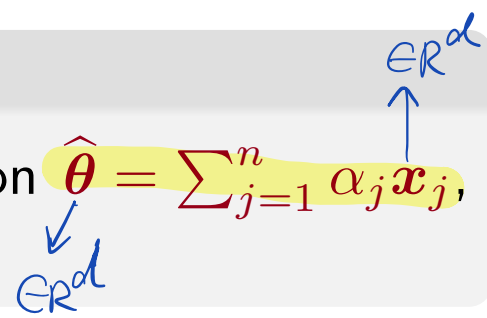


# Assumption of Kernel Method

- ▶ We will study a **trick** used to avoid the previous **computational issue** for lifting data to higher dimension, i.e., the so-called **kernel method**.

## Assumption on The Solution

We assume there exists  $\alpha \in \mathbb{R}^n$  such that the solution  $\hat{\theta} = \sum_{j=1}^n \alpha_j \mathbf{x}_j$ , where  $\{\mathbf{x}_j\}_{j=1}^n$  are training samples.



- ▶ With this assumption, instead of formulating learning problem over  $\theta \in \mathbb{R}^d$ , we can formulate learning problem over  $\alpha \in \mathbb{R}^n$  by replacing  $\theta = \sum_{j=1}^n \alpha_j \mathbf{x}_j$ .
- ▶ This assumption is mild as long as  $n \geq d$ , as likely training samples  $\{\mathbf{x}_j\}_{j=1}^n$  has  $d$  independent samples. In this case, the training data is easy to represent  $\theta$  using linear representation.

# Kernel Method / Trick

- In linear model, we always have  $\theta^\top x$ . Plugging  $\theta = \sum_{j=1}^n \alpha_j x_j$  into the linear model gives that many machine learning methods only involve the data through **inner products**

$$x^\top x',$$

where  $x, x'$  are two training samples.

$$\theta^\top x_i = \left( \sum_{j=1}^n \alpha_j x_j \right)^\top x_i = \sum_{j=1}^n \alpha_j \cdot \boxed{x_j^\top x_i} \quad \forall \begin{matrix} i=1, \dots, n \\ j=1, \dots, n \end{matrix}$$

# Kernel Method / Trick

- ▶ In linear model, we always have  $\theta^\top x$ . Plugging  $\theta = \sum_{j=1}^n \alpha_j x_j$  into the linear model gives that many machine learning methods only involve the data through **inner products**

where  $x, x'$  are two training samples.

- ▶ After applying the nonlinear transform  $\Phi$ , the inner product becomes

$$\begin{array}{c} x^\top x', \in \mathbb{R} \\ \downarrow \Phi \quad \downarrow \Phi \\ \Phi(x)^\top \Phi(x') \in \mathbb{R}. \end{array}$$

$\theta = \sum_{j=1}^n \alpha_j \Phi(x_j)$   
 $\theta^\top \Phi(x)$

- ▶ Kernel method is a trick to evaluate the above inner product **without explicitly calculating  $\Phi(x)$  and  $\Phi(x')$** .

## Kernel

A function  $\kappa: \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$  is a **kernel** for a nonlinear transform  $\Phi$  if

*computed in the original d-dim data space.*

$$\kappa(x, x') = \Phi(x)^\top \Phi(x'), \in \mathbb{R}, \forall x, x'$$

$\downarrow \mathbb{R}^d \quad \downarrow \mathbb{R}^d$

# Example of Kernel

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} \xRightarrow{\Phi} \Phi(\mathbf{x}) = \begin{bmatrix} 1 \\ x_1^2 \\ x_2^2 \\ \sqrt{2}x_1 \\ \sqrt{2}x_2 \\ \sqrt{2}x_1x_2 \end{bmatrix} \in \mathbb{R}^p$$

► The direct way of performing nonlinear transform:

- Compute  $\Phi(\mathbf{x})$  and  $\Phi(\mathbf{x}')$ .
- Compute  $\Phi(\mathbf{x})^\top \Phi(\mathbf{x}')$ .

► The kernel trick:

$$\kappa(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\top \mathbf{x}')^2 = \Phi(\mathbf{x})^\top \Phi(\mathbf{x}') \in \mathbb{R}$$

which equals to  $\Phi(\mathbf{x})^\top \Phi(\mathbf{x}')$ .

⇒ All the computations remain in dimension  $d$ . We never need to go to dimension  $p$ , i.e., compute  $\Phi(\mathbf{x})$ , explicitly.

# Valid Kernels

- ▶ In practice, we first define the kernel  $\kappa : \mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$ .
- ▶ How can we verify if there exists a  $\Phi$  such that

$$\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\top \Phi(\mathbf{x}') ?$$

→ There exists  $\Phi$  s.t.  $\kappa(\mathbf{x}, \mathbf{x}') = \Phi(\mathbf{x})^\top \Phi(\mathbf{x}')$

Mercer's theorem

$\kappa$  is a valid kernel if and only if the kernel matrix

$$\mathbf{K}(i, j) = \kappa(\mathbf{x}_i, \mathbf{x}_j)$$

is positive semidefinite for any set of data  $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ .

- ▶ In practice, we do not always care if there is a corresponding  $\Phi$  to the designed  $\kappa$ .
- ▶ Using  $\kappa$  with the best testing performance is often the guideline.

## Example: The Polynomial Kernel

- ▶ The polynomial kernel up to degree  $m$  is defined by

$$\kappa(\mathbf{x}, \mathbf{x}') = (1 + \mathbf{x}^\top \mathbf{x}')^m.$$

- ▶ We can always find a corresponding  $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$  consists of polynomials of degrees at most  $m$ .
- ▶ This can be verified by using binomial theorem.
- ▶ More generally, we have the polynomial kernel

$$\kappa(\mathbf{x}, \mathbf{x}') = (a + b\mathbf{x}^\top \mathbf{x}')^m$$

to adjust scale.

# Example: Gaussian / RBF kernel

Gaussian / Radial basis function (RBF) kernel:

$$\kappa(\mathbf{x}, \mathbf{x}') = \exp\left(-\frac{\|\mathbf{x} - \mathbf{x}'\|_2^2}{2\sigma^2}\right)$$

- ▶ The corresponding  $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$  has dimension  $p = \infty$ .
- ▶ This can be seen from Taylor's theorem. One dimensional example:

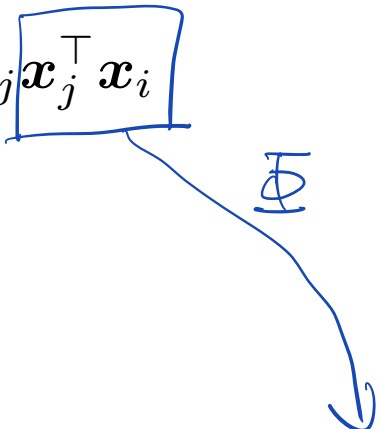
$$\kappa(x, x') = \exp(-(x - x')^2) = \exp(-x^2) \exp(x^2) \underbrace{\sum_{k=0}^{\infty} \frac{2^k x^k (x')^k}{k!}}_{\exp(2xx')}$$

- ▶ Hence, the RBF kernel can intuitively make any data linearly separable and regressible.

# How to Use Kernel Trick: The Process

- Pick/design a kernel  $\kappa$ .

- Change the trainable parameters  $\theta = \sum_{j=1}^n \alpha_j \mathbf{x}_j$ :

$$\theta^\top \mathbf{x}_i = \sum_{j=1}^n \alpha_j \boxed{\mathbf{x}_j^\top \mathbf{x}_i}$$


in loss function/decision rule.

- Applying the kernel trick: Replacing  $\mathbf{x}_i^\top \mathbf{x}_j$  with  $\kappa(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^\top \Phi(\mathbf{x}_j)$



# Learning Problems We Have Studied

General form:

$$\min_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \frac{1}{n} \sum_{i=1}^n \ell_i(\boldsymbol{\theta}).$$

- ▶ Least squares:  $\ell_i(\boldsymbol{\theta}) = (\boldsymbol{\theta}^\top \mathbf{x}_i - y_i)^2$ .
- ▶ Logistic regression:  $\ell_i(\boldsymbol{\theta}) = \log(1 + \exp(-y_i \boldsymbol{\theta}^\top \mathbf{x}_i))$ .
- ▶ SVM:  $\ell_i(\boldsymbol{\theta}) = \max(0, 1 - y_i \boldsymbol{\theta}^\top \mathbf{x}_i) + \lambda \boldsymbol{\theta}^\top \boldsymbol{\theta}$ .

If we invoke  $\boldsymbol{\theta} = \sum_{j=1}^n \alpha_j \mathbf{x}_j$ , we can 'kernelize' the above learning problems.

Let us take  $\ell_2$ -regularized LS as an example to invoke kernel trick.

# The Kernel Trick for Least Squares with Weight Decay

$$\hat{\theta} = \operatorname{argmin}_{\theta} \sum_{i=1}^n (\theta^{\top} \mathbf{x}_i - y_i)^2 + \lambda \|\theta\|_2^2$$

► We replace  $\theta$  by  $\theta = \sum_{j=1}^n \alpha_j \mathbf{x}_j$ .

Kernelization:

$$\hat{\alpha} = \operatorname{argmin}_{\alpha \in \mathbb{R}^n} \sum_{i=1}^n \left( \sum_{j=1}^n \alpha_j \mathbf{x}_j^{\top} \mathbf{x}_i - y_i \right)^2 + \lambda \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{x}_i^{\top} \mathbf{x}_j$$

$$\stackrel{\text{kernelize}}{=} \operatorname{argmin}_{\alpha \in \mathbb{R}^n} \sum_{i=1}^n \left( \sum_{j=1}^n \alpha_j \kappa(\mathbf{x}_i, \mathbf{x}_j) - y_i \right)^2 + \lambda \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \kappa(\mathbf{x}_i, \mathbf{x}_j)$$

$\kappa(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^{\top} \Phi(\mathbf{x}_j)$

$$= \operatorname{argmin}_{\alpha \in \mathbb{R}^n} \|K\alpha - \mathbf{y}\|_2^2 + \lambda \alpha^{\top} K \alpha$$

$$\kappa(i, j) = \kappa(\mathbf{x}_i, \mathbf{x}_j)$$

# The Kernel Trick for Least Squares with Weight Decay

$$\hat{\boldsymbol{\theta}} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^n (\boldsymbol{\theta}^\top \mathbf{x}_i - y_i)^2 + \lambda \|\boldsymbol{\theta}\|_2^2$$

► We replace  $\boldsymbol{\theta}$  by  $\boldsymbol{\theta} = \sum_{j=1}^n \alpha_j \mathbf{x}_j$ .

Kernelization:

$$\begin{aligned} \hat{\boldsymbol{\alpha}} &= \underset{\boldsymbol{\alpha} \in \mathbb{R}^n}{\operatorname{argmin}} \sum_{i=1}^n \left( \sum_{j=1}^n \alpha_j \mathbf{x}_j^\top \mathbf{x}_i - y_i \right)^2 + \lambda \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \mathbf{x}_i^\top \mathbf{x}_j \\ &\stackrel{\text{kernelize}}{=} \underset{\boldsymbol{\alpha} \in \mathbb{R}^n}{\operatorname{argmin}} \sum_{i=1}^n \left( \sum_{j=1}^n \alpha_j \kappa(\mathbf{x}_i, \mathbf{x}_j) - y_i \right)^2 + \lambda \sum_{i=1}^n \sum_{j=1}^n \alpha_i \alpha_j \kappa(\mathbf{x}_i, \mathbf{x}_j) \\ &= \underset{\boldsymbol{\alpha} \in \mathbb{R}^n}{\operatorname{argmin}} \|\mathbf{K}\boldsymbol{\alpha} - \mathbf{y}\|_2^2 + \lambda \boldsymbol{\alpha}^\top \mathbf{K}\boldsymbol{\alpha} \end{aligned}$$

Apply Kernel method for test data: With learned  $\hat{\boldsymbol{\alpha}}$  and a new test point  $\mathbf{x}$ , we have  $y = f(\mathbf{x}) = \sum_{j=1}^n \hat{\alpha}_j \kappa(\mathbf{x}_j, \mathbf{x})$ , which is non-linear in  $\mathbf{x}$ .

$$\hat{\boldsymbol{\theta}}^\top \boldsymbol{\Phi}(\mathbf{x}) = \left( \sum_{j=1}^n \hat{\alpha}_j \boldsymbol{\Phi}(\mathbf{x}_j) \right)^\top \boldsymbol{\Phi}(\mathbf{x}) = \sum_{j=1}^n \hat{\alpha}_j \underbrace{\boldsymbol{\Phi}(\mathbf{x}_j)^\top \boldsymbol{\Phi}(\mathbf{x})}_{=\kappa(\mathbf{x}_j, \mathbf{x})} = \kappa(\mathbf{x}, \mathbf{x}).$$

# Regularization Is Crucial for Kernel Methods

Kernelized problem:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmin}} \|\mathbf{K}\alpha - \mathbf{y}\|_2^2 + \lambda \alpha^\top \mathbf{K} \alpha$$

where  $\alpha \in \mathbb{R}^n$  and the kernel matrix  $\mathbf{K} \in \mathbb{R}^{n \times n}$ .

- ▶ What if  $\lambda = 0$  ?
- ▶ Do you remember  $p$ ?  $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$ . We often need  $p > n$ .

# Regularization Is Crucial for Kernel Methods

Kernelized problem:

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmin}} \|\mathbf{K}\alpha - \mathbf{y}\|_2^2 + \lambda \alpha^\top \mathbf{K} \alpha$$

where  $\alpha \in \mathbb{R}^n$  and the kernel matrix  $\mathbf{K} \in \mathbb{R}^{n \times n}$ .

- ▶ What if  $\lambda = 0$  ?
- ▶ Do you remember  $p$ ?  $\Phi : \mathbb{R}^d \rightarrow \mathbb{R}^p$ . We often need  $p > n$ .
- ▶ We are indeed in dimension  $p$ .
- ▶ Thus, the solution

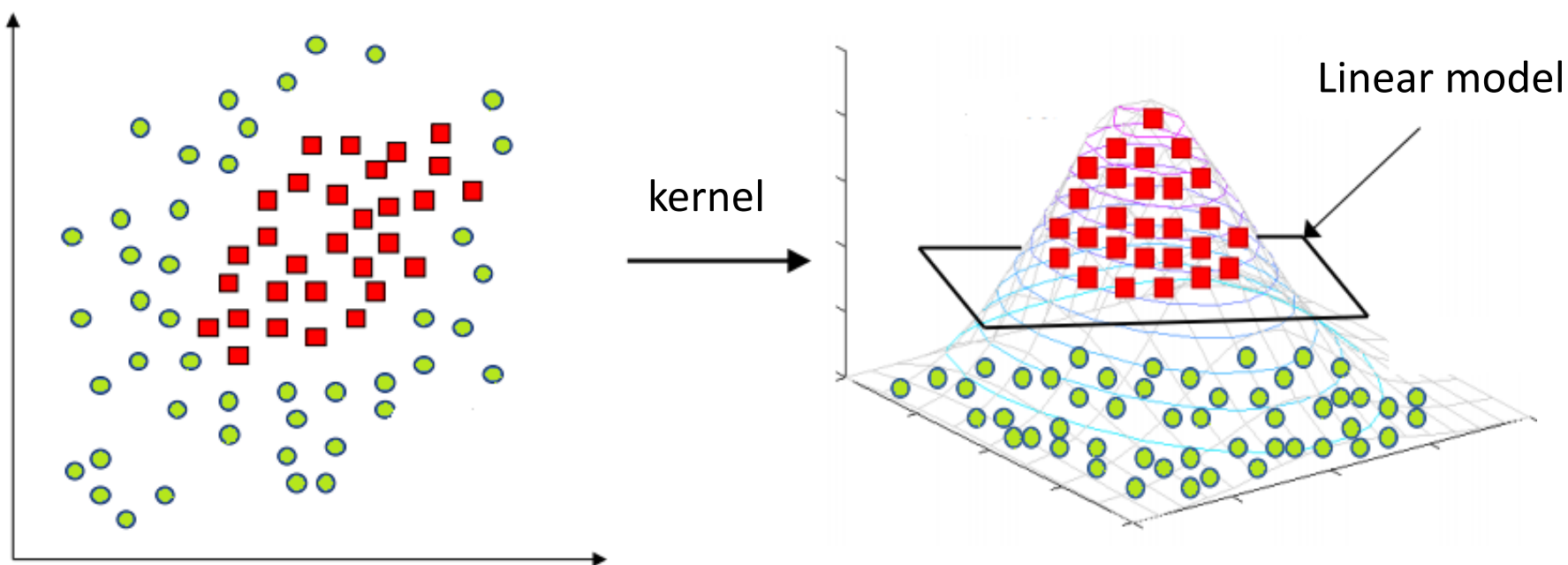
$$\hat{\alpha} = \mathbf{K}^\dagger \mathbf{y}$$

can **overfit** any data once  $p > n$ .  $\rightsquigarrow$  This is always the case where Gaussian / RBF kernel is used, as  $p = \infty$ .

- ▶ **Summary:** Kernel method is to use a as complex model as possible (with number of parameters  $p$ ), then we use regularization to penalize it to the right model that does not have severe overfitting.

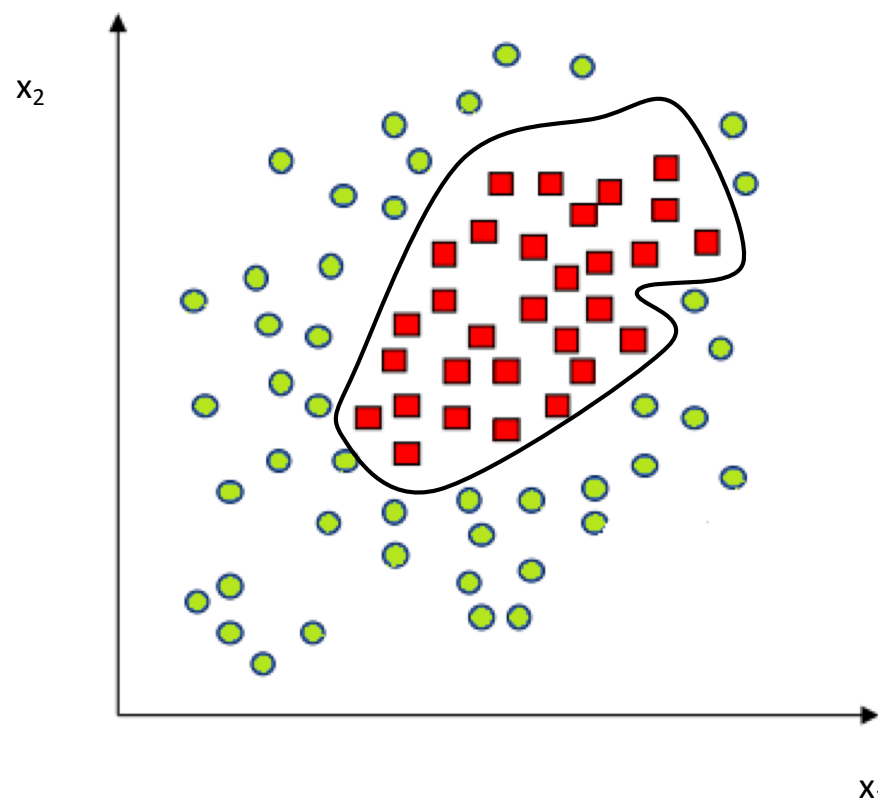
# Applying Kernel Methods to Calcification

In  $\Phi$  space: Linear in  $\theta$  and  $\Phi(x)$ .



# Applying Kernel Methods to Calcification

Apply kernel method for test data: With learned  $\hat{\alpha}$  and a new test point  $x$ , we have  $y \leftarrow f(x) = \sum_{j=1}^n \hat{\alpha}_j \kappa(x_j, x)$ , which is nonlinear in  $x$ .



Kernel method can be viewed as using linear models to learn nonlinear classifier / regressor by non-linearly transforming the data — An intermediate method between linear and nonlinear models.

~> Next lecture: Unsupervised learning.

## End of Linear Supervised Learning